Internal AI Knowledge Assistant Architecture Assessment, Scaling Strategy, and Cost Estimate

Overview

This document provides a formal technical assessment of an internal AI knowledge assistant developed for an air-conditioning maintenance company. The current implementation integrates a document folder stored in Microsoft SharePoint into a retrieval-augmented generation (RAG) workflow deployed on Azure AI Foundry, with BGE-M3 used as the embedding model and a Llama 3.3 60B-class model used as the core large language model.

The purpose of this document is fourfold: to evaluate whether the current model tier is sufficient when the knowledge base expands into additional SharePoint folders, to identify architecture optimization opportunities, to estimate the infrastructure required for an on-premise deployment option, and to provide a structured estimate of development, ongoing operation, and maintenance costs.

Current Architecture

The current solution follows a standard enterprise RAG design. Documents stored in SharePoint are ingested, chunked, embedded into vectors, indexed for retrieval, and passed to a generative model at inference time so that answers are grounded in enterprise content rather than model pretraining alone.

This design is appropriate for internal knowledge systems because it improves factual grounding, reduces hallucination risk, and enables updates to the knowledge base without retraining the foundation model.

Current stack summary

Layer	Current implementation	Assessment
Source repository	Microsoft SharePoint folder	Strong fit for organizations already using Microsoft 365 workflows
Retrieval architecture	RAG pipeline	Appropriate for document-grounded enterprise assistants

Embedding model	BGE-M3	Suitable for multilingual and varied enterprise document structures
Core LLM	Llama 3.3 60B-class	Strong reasoning tier for internal Q&A and synthesis
Deployment platform	Azure AI Foundry	Operationally efficient for Microsoft-centric environments

Functional flow

Step	Description	Key consideration
1. Ingestion	Pull documents from SharePoint folders	File freshness, access permissions, versioning
2. Preprocessing	Parse and split files into chunks	Technical manuals need structure-aware chunking
3. Embedding	Convert chunks into vector representations	Embedding quality affects recall performance
4. Indexing	Store searchable representations for retrieval	Metadata schema becomes more important as corpus grows
5. Retrieval	Retrieve candidate passages for a user question	Hybrid retrieval is preferred for technical content
6. Generation	LLM produces a grounded answer	Response quality depends on retrieval precision

Model Sizing for Expansion

Expanding from a single SharePoint folder to multiple folders does not automatically require a larger LLM. In production RAG systems, expansion usually causes accuracy issues first at the retrieval layer, including

lower precision, weak metadata filtering, overlapping content, outdated versions, and poor chunk structure.

For this reason, the current 60B-class model should be considered adequate for the next phase of expansion, especially if the future workload remains focused on internal document question answering, procedural assistance, and summarization rather than high-autonomy agentic workflows.

Model sizing guideline

Knowledge base scale	Recommended LLM tier	Suitable use case	Recommendation level
Small single-team repository	8B-14B	FAQ, SOP retrieval, simple support answers	Optional cost-down path
Multi-folder internal knowledge base	30B-70B	Cross-document synthesis, technical guidance, policy interpretation	Recommended tier
High-complexity multi-agent workflows	70B+	Long reasoning chains, tool orchestration, broader enterprise actions	Only if scope materially changes

Practical interpretation for this project

Scenario	Is model upgrade required?	Notes
Add more folders with similar document types	No, not by default	Retrieval improvements should come first

Add many document versions and overlapping content	Usually no	Reranking and metadata filtering are more important
Add highly complex reasoning or agentic actions	Possibly	Model size may need reassessment
Require lower operating cost with similar quality	Possibly downsize for some workloads	Smaller models can support lower-tier queries

Architecture Optimization Recommendations

The most effective optimization path is to strengthen the retrieval and governance layers rather than immediately increasing the LLM size. Technical maintenance content contains exact identifiers, model numbers, part names, abbreviations, and procedural references that benefit from more structured retrieval methods.

Optimization priorities

Priority	Recommendation	Why it matters	Expected impact
1	Hybrid retrieval	Combines vector search with lexical search for exact technical terms	Higher retrieval accuracy
2	Reranking	Reorders candidates to maximize relevance before generation	Better answer grounding
3	Metadata filtering	Limits retrieval by document type, model number, version, region, or business area	Lower noise and better governance
4	Structure-aware chunking	Preserves manuals, procedures, tables, and warning blocks	Better semantic coherence

5	Evaluation and observability	Tracks latency, retrieval quality, token usage, and grounded answer rate	Better reliability and cost control
6	Caching	Reduces repeated inference and accelerates common internal questions	Lower cost and faster response

Suggested target-state architecture

Layer	Current state	Recommended future state
Data source	Single SharePoint folder	Multiple folders with access-aware ingestion
Retrieval	Basic vector retrieval	Hybrid retrieval + metadata filtering + reranking
Preprocessing	Standard chunking	Structure-aware and document-type-aware chunking
Governance	Basic source grounding	Version-aware and permission-aware retrieval
Monitoring	Limited	Prompt, retrieval, latency, and usage monitoring

On-Premise Deployment Sizing

If the system is migrated to an on-premise environment, infrastructure requirements will rise substantially. Public hardware references for Llama 3.3 70B-class inference indicate that even 4-bit quantized deployment typically starts around 35 GB to 40 GB of VRAM, with additional headroom required for larger context windows, concurrency, and operational stability.

Because the current implementation is a 60B-class workload, planning assumptions should be aligned with the operational profile of other large 70B-class open-weight models. Low-concurrency pilot deployment may be feasible on a single high-memory GPU, but a production-grade enterprise deployment should target more capacity and redundancy.

Infrastructure sizing tiers

Tier	Suggested specification	Intended usage	Risk level

Pilot	1 x 40 GB GPU, 64 GB RAM, enterprise NVMe	Internal proof of concept, limited users	High operational risk
Production baseline	2 x 40 GB GPUs, 64-256 GB RAM, redundant storage/network	Stable internal assistant, moderate concurrency	Balanced recommendation
Growth-ready production	4 x 40 GB GPUs or equivalent, 256 GB+ RAM, orchestration stack	Larger user base, future upgrades, more simultaneous queries	Lower scale risk

Supporting infrastructure beyond GPUs

Component	Why it is required	Notes
CPU	Ingestion, parsing, indexing, and orchestration	LLM workloads are not GPU-only
System RAM	Vector DB, parsers, metadata services, and caching	Often underestimated in planning
NVMe storage	Fast index access, document staging, logs, and checkpoints	Important for retrieval performance
Monitoring stack	Reliability, alerting, capacity planning	Mandatory for production
Backup and failover	Service continuity and data protection	Required for enterprise use

Cost Framework

Cost should be evaluated across three phases: initial development, ongoing operation, and long-term maintenance. This is more useful than reporting only token charges because an internal enterprise assistant accumulates cost through integration work, platform services, governance, and support effort.

Cost categories

Cost category	Description	Typical examples
Development cost	One-time or project-based implementation effort	architecture design, SharePoint integration, chunking logic, prompt design, testing
Operating cost	Recurring service consumption	LLM tokens, search service, storage, monitoring, network
Maintenance cost	Ongoing support and improvement	index refresh, bug fixes, retrieval tuning, access review, model updates

Cloud Cost Estimate

Azure AI Search pricing indicates that service cost is based on selected SKU and search units, and public pricing examples show Standard S1 at about $245.28 per month and Standard S3 at about $981.12 per month in the referenced pricing page. Vector search itself does not incur a separate standalone fee beyond the underlying search service capacity, although storage, semantic ranker, and related features can affect total cost.

Azure AI Foundry pricing for Llama 3.3 70B shows a publicly listed global rate of about $0.71 per 1 million input tokens and $0.71 per 1 million output tokens, with regional and data zone variants slightly higher in some listings.

Example monthly cloud operating cost assumptions

The following table is an illustrative planning estimate rather than a vendor quote. It assumes a moderate internal usage profile for an enterprise knowledge assistant.

Cost item	Assumption	Estimated monthly cost (USD)	Source basis

LLM inference	40M input tokens + 10M output tokens	35.5	Llama 3.3 70B at $0.71/M input and output
Azure AI Search	Standard S1	245.28	Public pricing page
Storage and logs	Moderate internal usage	50-150	Planning allowance based on managed cloud usage model
Monitoring and observability	Azure monitoring services	30-100	Planning allowance for managed monitoring stack
Integration runtime / app hosting	Small internal application footprint	50-200	Planning allowance based on hosted application services
Total indicative monthly range	Excluding major custom support labor	410.78-730.78	Combined estimate

Example cloud operating cost scenarios

Scenario	Token usage pattern	Search tier	Indicative monthly platform cost
Small pilot	10M input + 2M output	Basic/S1 equivalent	About 150-400 USD depending on hosting mix
Moderate production	40M input + 10M output	S1	About 410-730 USD before internal labor
Larger internal rollout	150M input + 30M output	S2/S3 class	Roughly 1,200+ USD plus support overhead

On-Premise Cost Estimate

Public market references suggest that an H100 80GB PCIe GPU commonly falls in an approximate range of $25,000 to $30,000, while SXM variants often range from about $35,000 to $40,000. Full enterprise

servers with multiple H100 GPUs can exceed $100,000 and can reach above $200,000 depending on configuration.

Example on-premise capital cost ranges

Item	Low estimate (USD)	High estimate (USD)	Basis
2 x H100 80GB GPUs	50,000	80,000	Public H100 GPU pricing ranges
Server chassis, CPU, RAM, NVMe, networking	20,000	45,000	Derived from enterprise GPU server examples
Rack, power, cooling, setup overhead	10,000	25,000	Planning allowance for data center deployment
Total production-grade hardware baseline	80,000	150,000	Combined planning range

Example annualized on-premise operating and maintenance cost

Cost item	Example annual range (USD)	Notes
Electricity and cooling	6,000-15,000	Depends on utilization and local facility rates
Hardware support / warranty	4,000-12,000	Vendor and response-SLA dependent
Monitoring / platform software	2,000-10,000	Depends on toolchain maturity
Infrastructure administration	15,000-50,000	Internal labor allocation or managed support equivalent
Total indicative annual support range	27,000-87,000	Excluding major hardware replacement

Development Cost Estimate

A proper estimate should separate implementation effort from platform spend. For an internal RAG assistant integrated with SharePoint, development cost typically includes discovery, architecture design, ingestion pipeline work, retrieval tuning, prompt design, security alignment, testing, and deployment hardening.

Example development work breakdown

Work package	Indicative effort	Notes
Discovery and requirements definition	3-5 days	Business scope, document types, access rules
Architecture design	3-5 days	RAG flow, environments, service mapping
SharePoint ingestion and preprocessing	5-10 days	Parsing, chunking, update logic
Retrieval and indexing setup	4-8 days	Search schema, metadata, vector indexing
Prompt and answer workflow design	3-6 days	System instructions, grounding strategy
Evaluation and testing	5-8 days	Accuracy review, failure analysis
Deployment and hardening	3-6 days	Hosting, monitoring, security integration
Documentation and handover	2-4 days	Technical documentation and support guide

Example development cost model

The following table is an internal planning model using a consulting-style daily rate, not a market quote.

Team rate assumption	Effort range	Indicative one-time development cost (USD)
800/day	28-52 days	22,400-41,600
1,000/day	28-52 days	28,000-52,000
1,200/day	28-52 days	33,600-62,400

Maintenance Cost Estimate

Maintenance is often underestimated in enterprise AI projects because the system continues to evolve after go-live. The largest maintenance drivers are document growth, retrieval quality drift, prompt tuning, permissions changes, indexing failures, and user support.

Example monthly maintenance model

Maintenance activity	Example hours/month	Purpose
Index and ingestion monitoring	4-8	Ensure documents sync correctly
Retrieval tuning and review	4-10	Improve grounding quality
User issue handling	4-12	Resolve incorrect or missing answers
Security / access review	2-6	Maintain correct permission boundaries
Model or prompt adjustments	2-8	Improve response quality over time
Reporting and governance review	2-6	Usage trend and operational oversight
Labor rate assumption	Monthly support hours	Indicative monthly maintenance cost (USD)
---	---	---
80/hour	18-50 hours	1,440-4,000
120/hour	18-50 hours	2,160-6,000
150/hour	18-50 hours	2,700-7,500

Comparative Reading Aids

Cloud versus on-premise comparison

Dimension	Azure-hosted deployment	On-premise deployment
Initial cost	Low to moderate	High due to GPU hardware
Scalability	High	Limited by installed capacity
Maintenance burden	Lower platform burden	Significantly higher internal burden
Data control	Strong, but cloud-governed	Highest direct control
Time to deploy	Faster	Slower due to procurement and setup
Cost predictability	Moderate	Better after capital investment, but support costs remain

Recommended decision sequence

Decision question	Recommended answer for current stage
Is a larger LLM required immediately for more folders?	No, retrieval should be optimized first
What is the next technical priority?	Hybrid retrieval, reranking, metadata, and chunking
Is Azure still a suitable near-term platform?	Yes, especially while SharePoint remains the primary source system
When should on-premise be considered?	When data control, compliance, or sustained volume justify higher infrastructure ownership

Recommendation

The current architecture is technically sound, and the existing 60B-class LLM is sufficient for near-term expansion into additional SharePoint folders. The next phase should prioritize retrieval quality, metadata governance, reranking, structure-aware chunking, and observability rather than a larger generator model.

From a cost perspective, cloud deployment remains the more practical option for the current stage because it avoids high upfront hardware expenditure and reduces infrastructure maintenance overhead. On-premise deployment should be treated as a later strategic option and justified by governance, sovereignty, or sustained high-volume usage rather than by model performance alone.